Programming with Python



3. Lists

- ✓ What is a List?
- ✓ How to create and access?
- ✓ Intrinsic Methods of Lists
- ✓ List operations
- ✓ Nested Lists
- ✓ List Comprehension
- ✓ Examples

List

"Store one thing after another"

A list of students marks

A list of employee names

A list of movie names and year of release

List….

List is like a collection which allows us to put many objects in a single variable – Ordered and, we have an index to access

```
friends_list = [ 'Akhil', 'James', 'Amir' ]

travel_list = [ 'perfume', 'clothes', 'money' ]
```

Akhil	James	Amir
0	1	2

Perfume	Clothes	money
0	1	2

Create a list of movies

Three ways:

1. Create as a list constants
2. Create using list function
3. Create using list comprehensions

```
# 1. Create as a constant list
>>> movies = [ "Ice Age", "Gravity", "Rio"]
>>> print(movies)
[ 'Ice Age', 'Gravity', 'Rio']
```

Ice Age	Gravity	Rio
0	1	2

Create a list of movies…

```
# 2. Create using list function
# Create an empty list
>>> movies = list()
>>> print(movies)
[]
```

```
# Append the elements of the list
>>> movies.append("Ice Age")
>>> movies.append("Gravity")
>>> movies.append("Rio")
>>> print(movies)
['Ice Age', 'Gravity', 'Rio']
```

What is the data type of a list?

Is it required to declare the data type of a list?

No, Python variables don't have a type

"identifiers are simply names that refer to a data object *of some type*."

- Python needs to know that you need a list,
- you have given a name, and
- some data items are stored in it.

```
# Can we have different types of elements in a list?
>>> movies = ["Ice Age", 1, "Gravity", "Rio"]
>>> print(movies)
['Ice Age', 1, 'Gravity', 'Rio']
```

Yes

Accessing elements of a list (Indexing)

Printing a value using its index

```
>>> print(movies[0])
'Ice Age'
>>> print(movies[1])
'Gravity'
>>> print(movies[2])
'Gravity'
```

Out of bounds

```
>>> print(movies[5])
Index Error: list index out of range
```

Python do the bounds checking of lists

Negative Indexing

Printing a value using negative index

```
>>> print(movies)
['Ice Age', 'Gravity', 'Rio']
>>> print(movies[-1])
'Rio'
>>> print(movies[-2])
'Gravity'
>>> print(movies[-3])
'Ice Age'
```

Index → -1 refers to the last item, -2 refers to the last but one…

Slicing of Lists

Accessing a part of list using index range

```
>>> print(movies[0:2])  #Print movies[0], movies[1]
['Ice Age', 'Gravity']
```

```
>>> print(movies[:2])  #Print movies[0], movies[1]
['Ice Age', 'Gravity']
```

```
>>> print(movies[1:])  #Print movies[0], movies[1]
['Gravity', 'Rio']
```

```
>>> print(movies[:])  #Print movies[0], movies[1]
[''Ice Age', Gravity', 'Rio']
```

Movies[m : n] → Print the elements from the index 'm' to 'n-1'

Modifying a List

How to "modify the elements" of a list?

```
>>> movies[0]='007'
>>> print(movies)
['007', 'Gravity', 'Reo']
```

Can we modify using index range ???

```
>>> movies[1:3]=[] //Make 1st and 2nd elements as empty
>>> print(movies)
['007']
```

Properties of a list

1. **Heterogeneous** (any data type!)
2. Stored in **Contiguous**
3. Have **random access** to any element (Using index)
4. **Ordered** (numbered from 0 to n-1)
5. Number of elements can change very easily (use method .append)
6. Python lists are **mutable sequences** of **arbitrary objects**

Intrinsic Methods of Lists

Method	Description
append(elem)	Add item elem at the end of the list
extend(L)	Add all items in given list L to the end
insert(i, elem)	Insert item elem at position i
remove(elem)	Remove first item that is equal to elem, from the list
pop([i])	Remove and return item at position i (last item if i is not provided)
clear()	Remove all items and empty the list
index(elem)	Return index of first item that is equal to elem
count(elem)	Return the number of items that is equal to elem
sort()	Sort items in a list in ascending order
reverse()	Reverse the order of items in a list
copy()	Return a shallow copy of the list

List Methods

(a) Length of a List – len(d) - Return the number of items in the dictionary *d*.

```
>>> movies = [ "Ice Age", "Gravity", "Rio"]
>>> len(movies)
3
```

(b) Append element to a list

```
>>> movies.append('The Walk')
>>> movies
['Ice Age', 'Gravity', 'Rio', 'The walk']
```

(c) Extend a list

```
>>> movies.extend("Rio2", "Ice Age2")
>>> print(movies)
['Ice Age', 'Gravity', 'Rio', 'The walk', 'Rio2', 'Ice Age2']
```

List Methods…

(d1) Insert elements into a list at a desired position.

```
>>> movies = ["Ice Age", "Gravity", "Rio"]
>>> movies.insert(1, "Ice Age2")
>>> print(movies)
['Ice Age', 'Ice Age2', 'Gravity', 'Rio']
```

(d2) Insert more than one element at a desired position

```
>>> movies = ["Ice Age", "Gravity", "Rio"]
>>> movies.insert(1, "Ice Age2", "Ice Age3") //Wrong
>>> movies[1:1] = ("Ice Age2", "Ice Age3")
>>> print(movies)
['Ice Age', 'Ice Age2', 'Ice Age3''Gravity', 'Rio']
```

List Methods…

(e) Get the index of a given element (First occurrence..)

```
>>> movies = ["Ice Age", "Gravity", "Rio"]
>>> print(movies.index("Ice Age"))
0
```

(f) Count the frequency of an element

```
>>> movies = ["Ice Age", "Gravity", "Rio", "Ice Age"]
>>> print(movies.count("Ice Age"))
2
```

(g) Reverse a list

```
>>> movies = ["Ice Age", "Gravity", "Rio"]
>>> movies.reverse()
>>> print(movies)
['Rio', 'Gravity', 'Ice Age']
```

List Methods…

(h) Sort the elements of a given list

```
>>> movies = [“Ice Age”, “Gravity”, “Rio”]
>>> movies.sort()
>>> print(movies)
['Gravity', 'Ice Age', 'Rio']
```

(i) Pop an element from a list (Remove the last element)

```
>>> movies = [“Ice Age”, “Gravity”, “Rio”]
>>> movies.pop()
Rio
>>> print(movies)
['Ice Age', 'Gravity']
```



Removes the last element

List Methods…

(j) Clear all the elements from a list (Results into an empty list)

```
>>> movies = [“Ice Age”, “Gravity”, “Rio”]
>>> movies.clear()
>>> print(movies)
[]
```

(k) Delete a list

```
>>> movies = [“Ice Age”, “Gravity”, “Rio”]
>>> del movies[1]     //Deleting a single elem
>>> del movies[1:3]   // Deleting using slice
>>> del movies        // Delete completely
>>> print(movies)
NameError: name 'movies is not defined
```

What is the difference between “clear” and “del”??

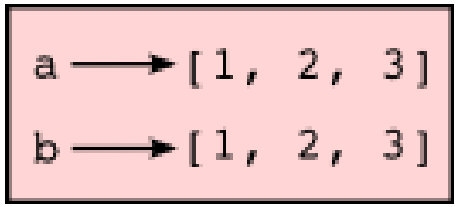
Cloning of Lists

How to create copies of a list?

1. Using explicit cloning – Using slice operator (:)
2. Using copy method

```
>>> a = [1, 2, 3]
>>> b = a[ : ]
>>> print(a, b)
[1, 2, 3] [1, 2, 3]
```

Taking any slice of a creates a new list. In this case the slice happens to consist of the whole list. So now the relationship is like this:



Objects and references - Lists

What about lists?

- a = [1 , 2, 3]
- b = [1 , 2, 3]

Are a and b pointing to the same list??

```
>>> a is b
False
```

a ⟶ [1 , 2, 3]

b ⟶ [1 , 2, 3]

So, “a” and “b” have same elements but are **not referring to the same object**.

Aliasing lists

Create an alias of a list

```
>>> a = [1, 2, 3]
>>> b = a
>>> a is b
True
```



So, “a” and “b” have same elements and **referring to the same object**.

List Membership

Check whether an element is a member of a list or not

- in
- not in

```
>>> a = [1, 2, 3]
>>> 1 in a
True
>>> 1 not in a
False
```

List Operations (+ , *)

Two operators:

- + To concatenate two lists
- * Repeats a list a given number of times

```
# Concatenation
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> a + b
[1, 2, 3, 4, 5, 6]
```

```
# Repeat
>>> a = [1, 2, 3]
>>> a * 2
[1, 2, 3, 1, 2, 3]
```

Lists and for loop

How to use for loop to parse a list?

```
for var in List:
        BODY of the LOOP
```

Example:

```
>>> friends = ["Ram", "Rahim", "John"]
>>> for friend in friends:
        print(friend)
```

For (every) friend in (the list of) friends, print (the name of the) friend.

Enumerate a list

enumerate generates pairs of both (index, value) during the list traversal.

Example:

```
>>> friends = [“Ram", “Rahim", “John"]
>>> for (i, v) in enumerate(friends):
                print(i, v)
0       Ram
1       Rahim
2       John
```

It generates key value pairs for a list of members.

Nested Lists

A list can be an element of another list

Example:

```
>>> nested = [“Ram”, “CDAC”, [1, 2, 3]]
>>> print(nested[2])
[1, 2, 3]
```

How to get the element from the nested list?

```
>>> n = nested[2]
>>> print(n[0])
1
```

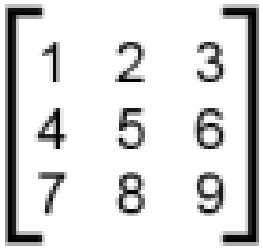
OR

```
>>> nested[2][0]
1
```

Matrices as a nested List

A matrix can be represented using a nested list.

Example:




```
>>> mx = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> print(mx[0])
[1, 2, 3]
>>> print(mx[1][2])
6
```

How to get the element from the nested list?

List Comprehension

- **List comprehensions provide a concise way to create new lists.**
- List comprehensions are a tool for transforming one list (any iterable actually) into another list.
- During this transformation, elements can be conditionally included in the new list and each element can be transformed as needed.

Example 1: Create a list of squares

```
>>> sq = [ ]
>>> for x in range(5):
.....sq.append(x**2)
>>> print(sq)
[0, 1, 4, 9, 16]
```

List Comprehension

List comprehensions provide a concise way to create new lists.

Example 1: Create a list of squares

```
>>> sq = [ ]
>>> for x in range(5):
.....sq.append(x**2)
>>> print(sq)
[0, 1, 4, 9, 16]
```

Better way…..???

List Comprehension

```
1 numbers = [1, 2, 3, 4, 5]
2
3 doubled_odds = []
4 for n in numbers:
5     if n % 2 == 1:
6         doubled_odds.append(n * 2)
7
8
~
```

Steps:

- Copy the variable assignment for our new empty list (line 3)
- Copy the expression that we've been append-ing into this new list (line 6)
- Copy the for loop line, excluding the final : (line 4)
- Copy the if statement line, also without the : (line 5)

List Comprehension

Let's use colors to highlight what's going on.

```
doubled_odds = [ ]
for n in numbers:
      if n % 2 == 1:
            doubled_odds.append(n * 2)
```

```
doubled_odds = [n * 2 for n in numbers if n % 2 == 1]
```

Examples of List Comprehensions

It is like building a series or a set in mathematics.

```
# Maths way
{x*x: x is a natural number in [0,100] }
{x : x is an even number less than 100 }
{ch: ch is a owel in word 'PYTHON' }
```

Python way - Comprehensions

```
[x**2 for x in range(0,101)]

[x for x in range(1,100) if x%2==0 ]

[x for x in 'PYTHON' if x in ['A','E','I','O','U']]
```

Nested List Comprehension

Nested List comprehensions can be worked on matrix like lists.

Example 1: Find out the transpose of a matrix

```
>>> matrix = [
... [1, 2, 3, 4],
... [5, 6, 7, 8],
... [9, 10, 11, 12],
... ]

>>> matrix = [[row[i] for row in matrix] for i in range(4)]

>>> print(matrix)
```

Output:

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Example2 – Calculate the prime numbers

Calculate the prime numbers between 1 to 100

```
>>> not_prime_list = [j for i in range(2, 8) for j in range(i*2, 100, i)]

>>> primes = [x for x in range(2, 100) if x not in not_prime_list]

>>> print(primes)
```

Output:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
```

Example3 – Celsius to Fahrenheit

Convert Celsius to Fahrenheit

```
>>> Celsius = [39.2, 36.5, 37.3, 37.8]

>>> Fahrenheit = [ ((float(9)/5)*x + 32) for x in Celsius ]

>>> print(Fahrenheit)
```

Output:

```
[102.56,     97.70000000000003,     99.14000000000001,
100.03999999999999]
```

For loop .vs. Comprehension

For Loop

```
for (set of values to iterate):
      if (conditional filtering):
            out_expr
```

Comprehensions

```
[ output_expr() for in set_of_values if cond ]
```

Summary

- ✓Introduction to Python
- ✓A simple program
- ✓Python Objects and Labels
- ✓Data types - Strict and dynamic Types
- ✓Memory Management
- ✓Operators and Expressions
- ✓Mutable and Immutable Objects

Thank You